

Game Recommendation Component — Viva Prep

AI-Powered Adaptive Learning Platform • Simple Q&A study sheet (English + Sinhala notes)

1. What the component does

Q: What does the game recommendation component do?

When a live class shows **sustained negative emotion** (bored / confused / frustrated), it alerts the teacher to "recommend a game" and suggests a **specific educational game** matched to the subject/lesson. The teacher **broadcasts** that game to all students, who play it, and the system **measures whether the negative emotion dropped afterwards**.

Sinhala: pantiy ෙ lamai digatama bored/confused unahama, teacher ට game ekak recommend karala, eka class ekatama broadcast karala, ee passe emotion eka adu unada kiyala maninawa.

Q: Where does it sit in the overall system?

- `emotion-service` (CNN, v5 model) — detects each student's emotion from webcam
- `emotion-backend` — stores emotions, **pattern detection, recommendation engine, intervention tracking** ← *this component*
- `frontend` — teacher dashboard alert + student "your teacher started a game" banner + the 13 games themselves

2. End-to-end flow (most important)

- 1 Student webcam → `emotion-service` CNN → emotion events → `emotion_store` in emotion-backend.
- 2 A **background tick every 30s** (`AGGREGATION_INTERVAL_SECONDS`) calls `aggregate_and_store()` → builds the **per-student class emotion distribution** (latest reading per student).
- 3 That distribution is fed to `pattern_detector.store_aggregation_result()`.
- 4 The pattern detector keeps, per negative emotion, an **unbroken "over threshold" streak timer**.
- 5 When a streak lasts $\geq$ `PATTERN_SUSTAINED_SECONDS` (default 600s = 10 min), `detect_dominant_pattern()` returns that emotion.
- 6 Teacher dashboard polls `/analytics/pattern` every 2s → shows the **"recommend a game"** alert.
- 7 Teacher clicks **Recommend** → `GET /recommendation/generate?emotion=&subject=&lesson_id=` → engine returns **1 primary game + 2 alternatives + a trigger_reason**.
- 8 The same call **starts an intervention**: `intervention_tracker.start_intervention()` snapshots the **pre-game emotion distribution**.
- 9 Teacher picks a game to broadcast → `POST /recommendation/active {game_key}` → creates a fresh **session_id**.
- 10 Every student's `ActiveGameBanner` polls `/recommendation/active` every 5s → shows *"Your teacher started a game → Play Now"*.
- 11 Student opens the game → `useGameSession` hook calls `/recommendation/active/join`; on win/loss it calls `/recommendation/active/finish` with outcome, score, correct/total, duration.
- 12 Teacher dashboard polls `/recommendation/active/stats` → live **joined count, finished count, per-student results table**.
- 13 After the game, post-emotion distribution → `POST /recommendation/intervention/{id}/feedback` → computes **negative-emotion reduction %** → `/recommendation/effectiveness` aggregates it.

3. Pattern detection (the trigger)

Q: What triggers a recommendation?

Not a single bad reading. A negative emotion must **exceed its threshold AND stay above it, unbroken, for 10 minutes**:

Emotion	Threshold (% of class)
BORED	> 30%
CONFUSED	> 25%
FRUSTRATED	> 20%

- Sustained bar: `PATTERN_SUSTAINED_SECONDS` = 600s default, **env-overridable, set low (e.g. 60s) for a demo**.
- If several emotions qualify at once, **most severe wins**: FRUSTRATED > CONFUSED > BORED.
- A threshold that **flickers on/off resets the streak** → no false alarms.

Q: Why sustained instead of instant?

A transient dip (someone looks away for 20s) isn't a real problem. 10 min = a **settled, real** class-level issue worth interrupting the lesson for.

Q: Why per-student distribution, not raw event count?

`aggregate_and_store()` uses **one latest reading per student** — the same number the teacher sees on the dashboard. A raw per-event count would over-weight whichever students' cameras poll most often.

Q: Why the background tick?

Earlier, the streak only advanced when someone happened to hit `/analytics/*`. Now a FastAPI **lifespan background task** re-aggregates every 30s, so "*sustained for 10 min*" is actually measured on its own.

4. The recommendation engine

Q: How does it pick a game?

`RecommendationEngine.generate_recommendation(emotion, subject, lesson_id) :`

1. Get all real games for the **subject** (currently Mathematics).
2. If a `lesson_id` is given and a game is **tagged to that lesson**, narrow to those; else fall back to the subject pool and set `lesson_matched=False` (honesty flag).
3. **Filter out game types used in the last 30 min** (`variation_window_minutes`) → repeated clicks cycle to something different.
4. If everything is filtered out, allow all again.
5. Never return the **exact same game as the last click** if another option exists.
6. `random.choice` from what's left → primary.
7. Build **2 alternatives** from other emotion buckets in the same subject.
8. Append to Redis history.

Q: Is the game matched to the detected emotion right now?

No — emotion→game mapping is intentionally OFF. By explicit design decision, the engine ignores the emotion bucket and cycles through **every real game for the subject**. Re-enabling is a one-line swap (`get_games_for(subject, emotion)` instead of `get_all_games_for(subject)`).

Q: Why is it off?

- Teacher-in-the-loop: the teacher decides, not the algorithm.
- Not enough real games per emotion bucket yet (would keep recommending the same 1–2).
- Avoids recommending a "game" that isn't actually built.

Q: What's the variation window?

30 minutes. The same **game_type** (e.g. "escape room game") won't be recommended twice inside that window, so the class gets variety.

Q: Why dedup by title, not game_id?

The same real game is listed under multiple emotion buckets with a **different game_id each time**. Dedup by `game_id` would treat them as different games and defeat the cycling.

5. Game catalog

Q: How many games? What subject?

13 real, playable games, all **Grade 9 Mathematics**, one per lesson — e.g. Fraction Room Rescue (fractions/BODMAS), Pirate Navigator (Pythagoras), Equations Eco (linear equations), Pattern Islands (number patterns), Fish Tank Shop (volume/capacity), Alchemist's Potion Lab (algebraic expressions), Time Guardians (rounding), Laser Heist (parallel-line angles), Fraction Chef, etc.

- `GAME_CATALOG` (recommendation engine) = the suggestion catalog, keyed `subject → emotion → [games]`. Other subjects (Science, English, History, Programming) still have **placeholder entries** — not built.
- `GAME_LIBRARY` (active broadcast) = the **13 that actually have a frontend route** and can be broadcast.
- Each game tagged with `lesson_id` linking it to a real lesson from adaptive-learning's `GET /api/lessons`.

Q: GameRecommendation fields?

`game_id`, `lesson_id`, `title`, `description`, `subject`, `game_type`, `difficulty`, `target_emotion`, `estimated_duration_minutes`, `engagement_score`.

6. Two recommendation paths (don't get confused)

	Emotion-driven suggestion	Manual teacher broadcast
Endpoint	<code>GET /recommendation/generate</code> , <code>/latest</code>	<code>POST /recommendation/active</code>
Who decides	Engine (from dominant emotion)	Teacher explicitly picks
Purpose	<i>Suggest</i> what to play	<i>Push</i> the game to all students
State	recommendation history + intervention	active session with join/finish counts

The teacher **first gets a suggestion**, then **broadcasts** a chosen game. They are separate stores.

7. Effectiveness / intervention tracking

Q: How do you know a game worked?

`InterventionTracker`:

- On recommend → save `pre_emotions` distribution.
- After the game → teacher/system posts `post_emotions` → `_calculate_reduction()`:
$$\text{reduction\%} = (\text{pre_negative} - \text{post_negative}) / \text{pre_negative} \times 100$$
where `negative` = `CONFUSED` + `BORED` + `FRUSTRATED` + `ANGRY`.
- `/recommendation/effectiveness` → **average reduction across interventions**, **target = 20%**, `target_met` flag, last 10 results.
- Redis-backed, keeps last 50 interventions.

Q: What if there were no negative emotions pre-game?

Returns `0.0` (can't divide by zero / nothing to reduce).

8. Frontend pieces

- `ActiveGameBanner.jsx` — global, in `__root.jsx`, students only. Polls `/recommendation/active` every 5s. Shows "Your teacher started a game" with route/label/badge from backend `GAME_LIBRARY`. Hidden if you're already on that game's page or you dismissed it.
- `useGameSession(gameKey)` — join/finish telemetry. If opened outside a live broadcast (direct link, no `session_id`), both calls are **no-ops**. Duration measured from page mount → result.
- `TeacherDashboard.jsx` — polls pattern every **2s** (proposal NFR: dashboard refresh $\leq 2s$), Recommend button, recommendation history log (last 7 days), live stats panel, "end game" control.
- **Stale-session guard** — every join/finish carries `session_id`; if it doesn't match the current broadcast (teacher already switched/ended), it's **silently ignored** so old results don't pollute the new session.

9. Design decisions & "why"

Decision	Why
Sustained 10-min streak, not instant	Only act on real, settled problems; no false alarms
Per-student distribution feeds detector	Don't over-weight students who poll more often
Background 30s tick	Streak advances on its own, not only when a route is hit
Emotion→game mapping OFF	Teacher-in-the-loop; too few real games per bucket; avoid suggesting non-existent games
30-min game-type variation window	Class gets variety, not the same game repeatedly
Dedup by title	Same game has multiple <code>game_ids</code> across emotion buckets
<code>lesson_matched</code> flag	Be honest when a suggestion isn't actually lesson-specific
Redis for state	Target production architecture: Redis = real-time/temporary state
<code>session_id</code> per broadcast	Scope join/finish to <i>this</i> run; ignore stale calls

10. Limitations / future work (be ready to admit)

- Only **Grade 9 Mathematics** games are real; other subjects are placeholder catalog entries.
- Recommendation is **not truly emotion-personalised** yet (mapping disabled) — it cycles through subject games.
- **Class-level**, not per-student — the whole class gets one game, not individualised.
- Effectiveness depends on **post-emotion feedback being submitted**; if the teacher skips it, no reduction number.
- Pattern detection is a **class aggregate** — a few struggling students can't trigger it.
- No long-term DB for intervention analytics (last 50 in Redis only).

11. Rapid-fire Q&A

- **Negative emotions list?** CONFUSED, BORED, FRUSTRATED, ANGRY.
- **Default sustained time?** 600s (10 min), `PATTERN_SUSTAINED_SECONDS`, env-overridable.
- **Aggregation interval?** 30s.
- **Variation window?** 30 min, on game *type*.
- **Effectiveness target?** 20% negative-emotion reduction.
- **How many games can be broadcast?** 13 (`GAME_LIBRARY`).
- **Tie-break between emotions?** FRUSTRATED > CONFUSED > BORED (most severe).
- **What resets a streak?** Emotion dropping below its threshold on any aggregation.
- **Where's state stored?** Redis (`recommendation:history`, `intervention:history`, `active_recommendation:state` ; pattern streaks in memory).
- **What connects a game to a lesson?** `lesson_id` tag matching adaptive-learning's `GET /api/lessons` .
- **What does the engine return?** primary game + 2 alternatives + `trigger_reason` + `lesson_matched` + `intervention_id`.
- **Emotion source?** `emotion-service` CNN (v5 model, deployed) via webcam.